

ARISTOTLE UNIVERSITY OF THESSALONIKI

Low-level Hardware Digital Systems II

System Verilog Assignment

by Dimitrios Latsinos

Department of Electrical and Computer Engineering
Major of Electronics and Computers

Contents

1 Introduction **1**

2 Module 1 - Main **1**
- 2.1 Sign, exponent and fractional part . . . 1
- 2.2 Normalization . . . 2
- 2.3 Pipelining . . . 2
- 2.4 Rounding . . . 2
- 2.5 Exception Handling . . . 2

3 Module 2 - Normalization **4**

4 Module 3 - Rounding **4**

5 Module 4 - Exception Handling **6**

6 Testbenching **7**
- 6.1 Results of first testbench . . . 8
- 6.2 Results of second testbench . . . 10

7 System Verilog Assertions (SVA) **13**

1 Introduction

The following constitutes my report for the assignment in the course "Low-level Hardware Digital Systems II", offered by the Department of Electrical and Computer Engineering at Aristotle University of Thessaloniki (AUTH) during the 8th semester of study. The project involved the implementation of a floating-point multiplier in accordance with the IEEE-754 arithmetic standard.

This document outlines the underlying logic used to write the System Verilog code, strictly adhering to the assignment's specifications. We will analyze the code describing the operation of the multiplier's distinct stages to clarify their logical interconnections and explain how they achieve the intended result. Furthermore, we will present and interpret the testbench results where necessary to verify that the output behaves as expected. Additionally, by utilizing assertions, we ensure that the finer functional details of the circuit are accurately captured in our implementation.

It should also be noted that Questa was used for the project, as recommended by the instructors. We provide screenshots of the simulation results, primarily focusing on the transcript output, as representing them in waveform format would be considerably difficult to read.

2 Module 1 - Main

First, we present the main multiplier circuit, which consists of the submodules detailed in subsequent sections. The corresponding .sv file is "fp_mult.sv". The logic diagram of the multiplier is shown in "Figure 1", located at the end of this paragraph.

The System Verilog module describing it shares the file's name, "fp_mult". As seen from its definition, it features the primary inputs depicted in Figure 1": two 32-bit floating-point numbers, a" and b", and the argument indicating the rounding scheme. In addition, it incorporates a "reset" signal and a clock signal, which are utilized by the pipeline registers.

2.1 Sign, exponent and fractional part

Our first priority is to handle the operations between the individual components that make up floating-point numbers. As specified in the assignment description, these numbers consist of the sign, the exponent, and the mantissa (fraction).

The sign of the product is computed using an XOR logic gate, with the signs of the two operands serving as its inputs.

Next, we calculate the sum of the exponents, which is initially normalized by subtracting the bias of 127. In this way, we obtain the exponent of the product to be used in the 32-bit representation. Subtracting 127 twice would yield the "true" exponent rather than its IEEE-754 biased representation. It is also important to note that we extend the exponents by two bits to better handle potential overflow, as will be discussed later. The term "sum_of_exponents" is declared as signed, since it can take negative values.

The next step is the multiplication of the mantissas, including the implicit leading "1" of the significand.

2.2 Normalization

What follows is normalization, which yields the normalized exponent and the normalized mantissa.

Into the instance of the "normalize_mult" module, we feed the previously computed sum of exponents and the product of the mantissas. We also capture the guard and sticky bit outputs—which will be utilized during rounding—as well as the normalized exponent and mantissa (having declared them as logic types).

2.3 Pipelining

The implementation also requires a pipelining scheme that enables the simultaneous intake of new data, processing of earlier stages, and output of final results within the same clock cycle. In this manner, a new product for different inputs can potentially be computed on every cycle, as demonstrated in the testbenches.

For this purpose, we create a package containing a packed struct type named "pipeline_data_t". This struct is utilized inside an always_ff block that registers the data from the preceding stage on every cycle and feeds it into the subsequent submodules for rounding and exception handling. Therefore, we must store:

- the inputs "a" and "b"
- the sign bit "sign"
- the normalized exponent and mantissa
- the "guard" and "sticky" bits
- the selected rounding mode "rnd" (whose type is defined in a separate package)

2.4 Rounding

Next is the instantiation of the "round_mult" module, which yields the exponent and mantissa after rounding, should rounding be deemed necessary. From the contents of the pipeline registers, we supply the normalized mantissa along with the hidden integer bit "1", the normalized exponent, the desired rounding scheme, the guard and sticky bits, and the sign bit. In addition to the updated exponent and mantissa, we declare an inexact" output bit, which indicates whether rounding occurred depending on whether it is driven "high" or "low".

2.5 Exception Handling

The final stage addresses exception handling—namely, cases where bypassing or overriding the preceding stages is necessary to produce the correct result according to the IEEE-754 standard.

Before instantiating this module, we will outline how the result is assembled after the rounding module and explain how overflow and underflow are signaled.

For the former, we take the sign bit (stored in the pipeline registers) along with the post-rounding exponent and mantissa. Using an assign statement, we concatenate them to form a 32-bit floating-point number (since the post-rounding exponent has a width of 10 bits, we retain its 8 LSBs).

Overflow and underflow are represented using two 1-bit signals named “overflow” and “underflow”, respectively, which assert high when these conditions occur. We consider an overflow to have occurred when the post-rounding exponent exceeds the exponent of MaxNorm (i.e., greater than 254), whereas underflow occurs when the exponent immediately following normalization is strictly less than that of MinNorm (i.e., less than 1).

To instantiate the exception handling module (referred to here in conjunction with the multiplier pipeline), we feed in the registered inputs “a” and “b”, the rounding scheme “rnd”, the “overflow” and “underflow” flags, the “inexact” signal, and the intermediate result from the rounding module. The outputs are designated as the final multiplication result “z” and 6 of the 8 status bits, which convey status flags regarding the arithmetic execution and the resulting values. These 6 flags are “zero_f”, “inf_f”, “nan_f”, “tiny_f”, “huge_f”, and “inexact_f”. The flag “div_by_0_f” asserts high if “inexact_f” is low, “inf_f” is high, and both operands “a” and “b” are finite numbers (their exponents do not consist entirely of all 1s). Finally, an “assign” statement bundles these flags together to form the “status” vector.

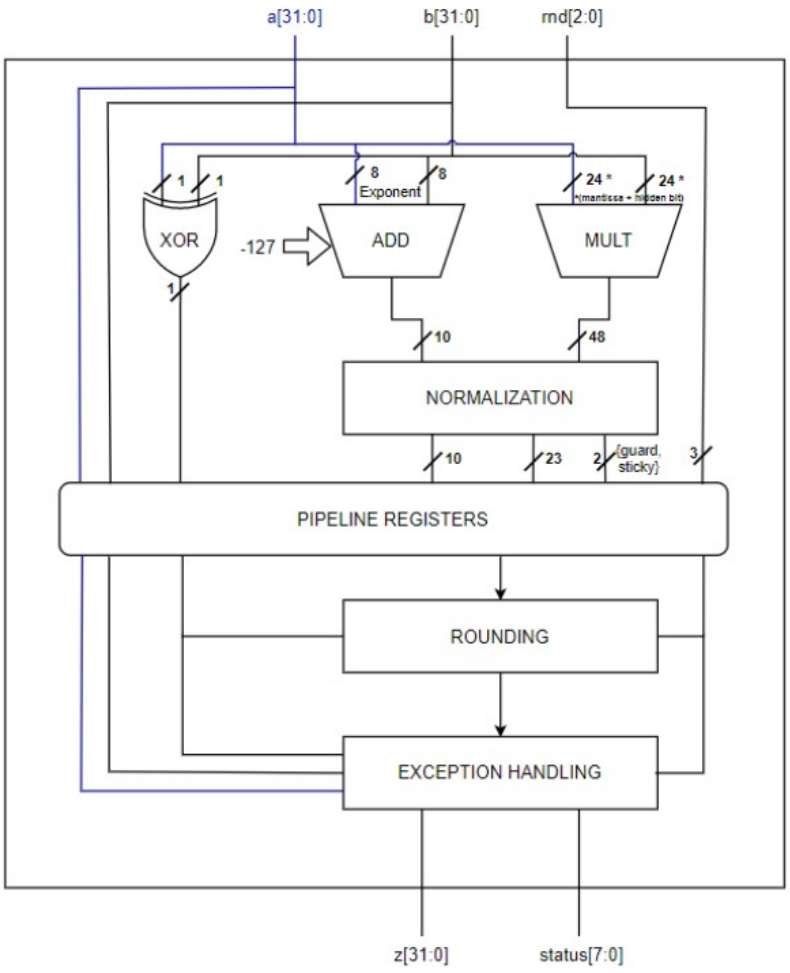


Figure 1: Logic Diagram of Floating-Point Multiplier

3 Module 2 - Normalization

Next, we proceed with the presentation of the submodules utilized by the multiplier circuit to handle floating-point numbers. First, we examine “normalize_mult”, which is defined in the file of the same name.

As mentioned earlier, its inputs are the 48-bit product of the significands (including the implicit leading 1s) and the sum of the exponents minus the bias (127). Its outputs are the guard and sticky bits, the normalized mantissa, and the normalized exponent.

Since this constitutes purely combinational logic, an always_comb block suffices. Consequently, throughout the simulations, its outputs can be evaluated instantaneously, adhering to the structure shown in the following figure.

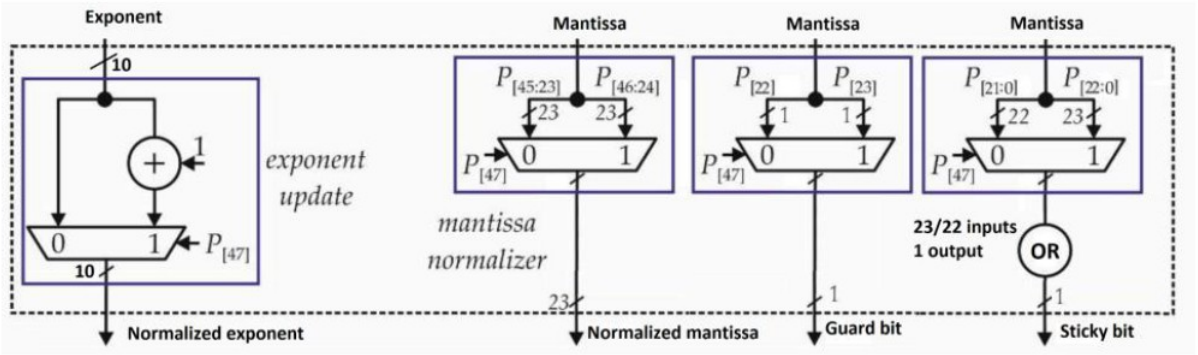


Figure 2: Normalization module logic diagram

We compute the normalized exponent based on the value of the product’s MSB. If it is 1, we increment the biased sum of exponents by one; otherwise, it remains unchanged.

For the normalized mantissa, the MSB of the product dictates which slice of bits is selected. If the MSB is 1, we select bits of the product; otherwise, we select bits.

Following the same rationale, we determine the guard bit. If the product’s MSB is 1, the guard bit is bit 23 of the product; otherwise, it is bit 22.

Finally, we determine the sticky bit, which indicates whether a logic 1 appears in any position following the guard bit. For this purpose, we apply a reduction OR operation across the bits trailing the guard bit.

4 Module 3 - Rounding

At this stage, we demonstrate the operation of the rounding submodule as defined in “round_mult.sv”. To reiterate, the inputs are the normalized mantissa, the normalized exponent, the rounding mode, the guard and sticky bits, and the sign of the product. The outputs are the post-rounding mantissa and exponent, along with the inexact bit.

To begin with, we define a package named "round_pkg", which contains an enum logic type named "round". This is used to encode the different rounding modes, which are the following:

- IEEE_near, which rounds to the nearest representable value and, in the event of a tie, selects the one with an even significand (round to nearest, ties to even)
- IEEE_zero, which always rounds toward zero (truncation)
- IEEE_pinf, which always rounds toward $+\infty$ (round up)
- IEEE_ninf, which always rounds toward $-\infty$ (round down)
- near_up, a standard rounding mode that rounds to the nearest representable number, breaking ties toward $+\infty$
- away_zero, which rounds away from zero to the value with the larger magnitude

In this module as well, the circuit consists of purely combinational logic, so we again employ an always_comb block.

Initially, a check is performed: if both the guard and sticky bits are 0, no rounding is required and the inexact flag is set to 0. Otherwise, the inexact flag is set to 1, indicating that rounding is necessary. Next, we analyze the implementation logic for each rounding scheme, which is handled via a case statement evaluated on the encoded rounding mode.

For IEEE_near, we branch based on the combination of guard and sticky bits. If the guard bit is 1 and the sticky bit is 0, a tie occurs. Thus, we check whether the mantissa is even; if so, the mantissa remains unchanged, whereas if it is odd, it is incremented by 1 to select the next representable number. If the guard bit is 0 and the sticky bit is 1, we round down (retaining the same mantissa), whereas if both guard and sticky bits are 1, we round up (incrementing the mantissa by 1).

Under the IEEE_zero mode, the mantissa is always kept as is (i.e., truncating the remaining bits).

Under IEEE_pinf, the decision depends on the sign. If the result is positive, we round up (incrementing by 1); otherwise, we preserve the current mantissa.

For IEEE_ninf, the opposite logic applies: if the result is positive, we round down (preserving the current mantissa), whereas if negative, we increment by 1.

For the near_up mode, the behavior accounts for directional ties. If the number is negative, a tie leaves the mantissa unchanged, while non-tie cases round to the nearest representable number following the same logic as IEEE_near. Conversely, if the number is positive, a tie causes the mantissa to increment by 1, with non-tie cases again rounding to nearest.

When using the away_zero mode, the mantissa is unconditionally incremented by 1.

Additionally, IEEE_near is set as the default case.

Finally, we inspect the MSB of the mantissa after rounding addition. If overflow occurs (the MSB is 1), the mantissa is shifted right by one bit and the exponent is incremented by 1. Otherwise, both the exponent and mantissa are retained as they are.

5 Module 4 - Exception Handling

The final part of the multiplier is the module used for exception handling. This is defined in the file "exception_mult.sv". We mention again that it has as inputs "a" and "b", the "z_calc" that has been calculated so far, the signals that warn of overflow, underflow and prior rounding, and the rounding scheme. As outputs, we have the final output of the multiplier and 6 of the 8 status bits.

Before implementing a case that will cover every exception, we will write two functions to facilitate our work. One is "num_interp" and maps a 32-bit signal to one of the values that an enum interp_t can take. This enum describes the different cases for inputs "a" and "b". These are the cases:

- ZERO, in which zero and the denormals are included
- INF, where infinity and the NaNs (signaling and quiet) belong
- NORM, where the normals are classified
- MAX_NORM, i.e. the maximum normal
- MIN_NORM, i.e. the minimum normal

To ZERO are mapped the signals with zero exponent, to INF those with exponent consisting only of 1s, and to NORM all the rest.

The second function is called "z_num" and does the reverse, namely it maps the ZERO and INF categories to their 31-bit values according to IEEE-754 (the sign is not taken into account).

By defining two types "interp_t", which we make sure to always indicate which category the inputs of the multiplier belong to with "num_interp" . This assignment is done using an always_comb.

In a second always_comb, we will define the case for all exceptions. At its beginning, the status bits are initialized with the value 0.

For the case where both inputs of the multiplier are considered ZERO (they are either actually zero or denormals), the final result is zero (with the sign of "z_calc") and the value 1 is assigned to "zero_f".

Similarly, for the case of one ZERO and one NORM.

If we have one ZERO and one INF, the result is considered infinity (with the sign of "z_calc") while the value 1 is assigned to "inf_f". In case the MSB of the mantissa of "z_calc"is equal to 0, the value 1 is also assigned to "nan_f" (it is considered signaling NaN; otherwise, it is quiet).

For two INFs, we simply get infinity (with the sign of "z_calc") and "inf_f" equal to 1.

For one INF and one NORM, again infinity (with the sign of "z_calc") and "inf_f" equal to 1.

The last scenario is the case where we have two NORMs. Here we act depending on whether we have overflow, underflow, or neither of the two.

If we have overflow, initially "huge_f" and "inexact_f" are set equal to 1. Then, we distinguish cases according to the rounding we follow. For IEEE_near, we simply round to infinity. When we have IEEE_zero, we round to the maximum normal. If we have IEEE_pinf, we round to the maximum normal for negative numbers, while for positive we round to infinity. The opposite for the case of IEEE_ninf. For the near_up standard and away_zero, we always round to infinity. In every case where the result is infinity, we assign the value 1 to "inf_f".

When we have underflow, “tiny_f” and “inexact_f” are set equal to 1. Again, based on rounding, we get a different result. For IEEE_near, we simply round to zero. When we have IEEE_zero, we round again to zero. If we have IEEE_pinf, we round to zero for negative numbers, while for positive we round to the minimum normal. The opposite for the case of IEEE_ninf. For the near_up standard, we always round to zero. And finally, for away_zero, we round to the minimum norm. In every case where the result is zero, we assign the value 1 to “zero_f”.

And if the conditions of overflow and underflow do not hold, we simply keep “z_calc” for output, while “inexact_f” takes the value of “inexact”.

6 Testbenching

Having implemented the multiplier, our main priority now is to perform testbenches that will demonstrate its correct operation. For this purpose, the instructors provided a function that gives the expected result of multiplying two numbers given the rounding scheme being followed, as well as a wrapper module “fp_mult_top”. For the first testbench, we are required to verify that for random arguments, the multiplier we built produces the same results as the multiplication function, whereas in a second testbench we will verify that in the presence of exceptions the result is again the expected one. Both were implemented in the file “testbench.sv”.

Let us discuss, first, the essential components of the first testbench. The most critical is the design of a structure that will print the output of the modules and the result of the multiplication function concurrently, despite the fact that in the former case 3 cycles are required to generate the result, while in the latter it occurs instantaneously. For this purpose, we will define a new struct that will store the result of the multiplication function, the two numbers, and the rounding scheme. Next, we will define a queue in which such struct instances will be stored. In this way, after the proper number of cycles for alignment, we will print the function result simultaneously with the multiplier result. In addition, we define two struct types: one for queue entry and one for queue removal/printing of results.

We first define the arguments of “fp_mult_top”, among which are the clock and the reset signal (rst). With an always block, we take care of toggling the clock every 5 ns, resulting in a positive edge trigger every 10 ns. We also define an enable signal which, when it is “high”, signals that we are still in the first testbench, while when it is “low”, we are in the second. This will assist us in the second always block, which will feature two if statements: one fulfilled when we are in the first testbench and the reset signal is “high”, and the other when we are in the second testbench and the reset is high.

We will now describe the first if for the first testbench. On every positive edge of the clock, if the conditions mentioned above are met, pseudo-random values are assigned to “a” and “b” using the $urandom() function. Utilizing the struct for queue insertion that we implemented earlier, we push everything we need. Next, a check is performed: if we have more than 4 structs in the queue, we pop the first one that entered. From this, we write all fields via an $fdisplay() to a file “tb_log_fd” along with the multiplier result. The contents of this file will be printed at the end. Let us now consider why this works: on each new cycle, “a” and b” take new values, and therefore new structs are created. One cycle after the first, we will have 2 structs in the queue and the first ones will be assigned to the instance of “fp_mult” contained inside “fp_mult_top”. After two cycles, we will have storage in the pipeline registers and three structs. After three cycles, we will have storage of the output in “fp_mult_top” (as desired) and four structs. However, due to how we have written the always block, the values of z would be printed prior to the new assignment. Thus, we need one more cycle, bringing the structs to 5, which is greater than 4. In this way, we achieved the alignment of the function results and the multiplier results. This means that chronologically they would be printed one cycle later, had we not subtracted 10 ns from the current time.

We now move to the initial block for the first testbench. Initially, we open the file “tb_log_fd” to write to it whatever we wish to print. At the start we set enable equal to 1, the clock equal to 1, initialize the function rounding scheme equal to the string for IEEE_near, reset equal to 0, and rnd such that it agrees with the encoding of IEEE_near. Initially, we have zeroing of the registers and an empty queue. After 11 ns, we set reset equal to 1, so that we can normally utilize the contents of the always block. On every positive edge, as stated, we take new “a” and “b” and everything proceeds as described above. Thus, the first pair will be applied at 20ns and the result at 50ns. It is reiterated that everything is logged to the .txt file that will be printed at the end.

Having allowed sufficient time to verify 45 multiplications (490 ns), we proceed to testing a different rounding scheme. We reset the registers and empty the queue once more. We change the arguments of the function and the multiplier for the IEEE_zero rounding scheme and deassert reset back to logic 1 after the appropriate delay (10 ns). The above process is then repeated for all rounding schemes. This concludes the analysis of the first testbench.

For the second testbench, now, we must test all the different corner cases (144 combinations). We define an enum “corner_case”, which encodes all the numbers that constitute the edge cases. Then, we define a function “match_cs_to _bin”, which will return a random number (except in cases where it is fixed) belonging, for instance, to the negative or positive signaling NaNs, etc., provided we pass as an argument the category to which it belongs. We also define two struct enums of type “corner_case”, which we will utilize in the always block. The if for the second testbench does not differ substantially from the one for the first, except only in that “a” and “b” are generated differently and in that it operates when enable is “low”.

Following the completion of the first testbench, with appropriate delays we once again apply reset, empty the queue, and change enable to 0. Subsequently, we take all the different combinations that can arise and verify their results in the exact same manner. This is accomplished by picking one boundary case and testing all its combinations against all cases. This is repeated 12 times, since we have 12 edge cases. The order of combinations follows their presentation order in the assignment prompt. Thus, if we take, for example, all combinations with positive zero, we start with negative signaling NaN, followed by positive signaling NaN, then negative quiet NaN, etc.

The total simulation time is equal to 5151 ns and the simulation settings were identical to those specified in the project assignment (enabled optimization, full visibility, etc.).

In the following pages we will present some simulation results, and demonstrate that the multiplier value matches that of the multiplication function across all cases. The use of the submitted files is also suggested for a more direct demonstration.

6.1 Results of first testbench

```
# Rounding Case : IEEE_near
# ================================================================================================
# COMPARE @50 | a=8d3d0772 b=264ea0b2 | DUT=80000000 | status=29 | REF=80000000 | MODE=IEEE_near
# COMPARE @60 | a=a3c56e10 b=0f19f530 | DUT=80000000 | status=29 | REF=80000000 | MODE=IEEE_near
# COMPARE @70 | a=8b4de7b5 b=5eca6db6 | DUT=aaa2d113 | status=20 | REF=aaa2d113 | MODE=IEEE_near
# COMPARE @80 | a=b0a9cb14 b=5be49cdf | DUT=cd17a0e9 | status=20 | REF=cd17a0e9 | MODE=IEEE_near
# COMPARE @90 | a=8456aebb b=45e33beb | DUT=8abe8f2f | status=20 | REF=8abe8f2f | MODE=IEEE_near
# COMPARE @100 | a=e27e4441 b=2b4e4040 | DUT=ce4cdabd | status=20 | REF=ce4cdabd | MODE=IEEE_near
# COMPARE @110 | a=9dd2f18a b=90c4e303 | DUT=00000000 | status=29 | REF=00000000 | MODE=IEEE_near
# COMPARE @120 | a=a64e55fc b=a4ff945d | DUT=0bcdff3b | status=20 | REF=0bcdff3b | MODE=IEEE_near
# COMPARE @130 | a=8acbc21d b=72612a6a | DUT=bdb3375e | status=20 | REF=bdb3375e | MODE=IEEE_near
# COMPARE @140 | a=1873b116 b=0737353f | DUT=00000000 | status=29 | REF=00000000 | MODE=IEEE near
```

Figure 3: 1ο testbench : IEEE_near (10 συνδυασμοί)

```
# Rounding Case : IEEE_zero
# ================================================================================================
# COMPARE @550 | a=43597a37 b=87b2e1b5 | DUT=8b97f6b8 | status=20 | REF=8b97f6b8 | MODE=IEEE_zero
# COMPARE @560 | a=65630670 b=519776ad | DUT=7786520a | status=20 | REF=7786520a | MODE=IEEE_zero
# COMPARE @570 | a=8205b302 b=dce856f7 | DUT=1f72af49 | status=20 | REF=1f72af49 | MODE=IEEE_zero
# COMPARE @580 | a=f86cc698 b=cd630a81 | DUT=7f7fffff | status=30 | REF=7f7fffff | MODE=IEEE_zero
# COMPARE @590 | a=9112e35a b=8292ed51 | DUT=00000000 | status=29 | REF=00000000 | MODE=IEEE_zero
# COMPARE @600 | a=cde00ba2 b=097122f7 | DUT=97d3098d | status=20 | REF=97d3098d | MODE=IEEE_zero
# COMPARE @610 | a=9f58de70 b=d3a823ae | DUT=338e7033 | status=20 | REF=338e7033 | MODE=IEEE_zero
# COMPARE @620 | a=67cce4d8 b=02a3ff5d | DUT=2b034217 | status=20 | REF=2b034217 | MODE=IEEE_zero
# COMPARE @630 | a=b28dda75 b=0a15255e | DUT=80000000 | status=29 | REF=80000000 | MODE=IEEE_zero
# COMPARE @640 | a=e154423e b=bdb7b414 | DUT=5f9850a9 | status=20 | REF=5f9850a9 | MODE=IEEE_zero
```

Figure 4: 1ο testbench : IEEE_zero (10 συνδυασμοί)

```
# Rounding Case : IEEE_pinf
# ================================================================================================
# COMPARE @1050 | a=06086a8c b=0a452552 | DUT=00800000 | status=28 | REF=00800000 | MODE=IEEE_pinf
# COMPARE @1060 | a=f1040788 b=0b9c678a | DUT=bd2153fa | status=20 | REF=bd2153fa | MODE=IEEE_pinf
# COMPARE @1070 | a=b5043063 b=d17efd19 | DUT=4703aab3 | status=20 | REF=4703aab3 | MODE=IEEE_pinf
# COMPARE @1080 | a=113b4cab b=32f884d1 | DUT=04b5d373 | status=20 | REF=04b5d373 | MODE=IEEE_pinf
# COMPARE @1090 | a=159405c3 b=0b217fdc | DUT=00800000 | status=28 | REF=00800000 | MODE=IEEE_pinf
# COMPARE @1100 | a=76675df5 b=6dab64c0 | DUT=7f800000 | status=32 | REF=7f800000 | MODE=IEEE_pinf
# COMPARE @1110 | a=a871cd50 b=f999c103 | DUT=62913a04 | status=20 | REF=62913a04 | MODE=IEEE_pinf
# COMPARE @1120 | a=e50db106 b=d422fe18 | DUT=79b46d52 | status=20 | REF=79b46d52 | MODE=IEEE_pinf
# COMPARE @1130 | a=9ded8447 b=7598875b | DUT=d40d841f | status=20 | REF=d40d841f | MODE=IEEE_pinf
# COMPARE @1140 | a=00181c1f b=87a47864 | DUT=80000000 | status=01 | REF=80000000 | MODE=IEEE_pinf
```

Figure 5: 1ο testbench : IEEE_pinf (10 συνδυασμοί)

```
# Rounding Case : IEEE_ninf
# ================================================================================================
# COMPARE @1550 | a=c0c066c7 b=68f1116f | DUT=ea352ddc | status=20 | REF=ea352ddc | MODE=IEEE_ninf
# COMPARE @1560 | a=b114a9aa b=014a8686 | DUT=80800000 | status=28 | REF=80800000 | MODE=IEEE_ninf
# COMPARE @1570 | a=bf8bacdb b=afb2582a | DUT=2fc29c95 | status=20 | REF=2fc29c95 | MODE=IEEE_ninf
# COMPARE @1580 | a=1e2b06fa b=47cfa209 | DUT=268ab6e4 | status=20 | REF=268ab6e4 | MODE=IEEE_ninf
# COMPARE @1590 | a=f8ad7a27 b=cb898e90 | DUT=7f7fffff | status=30 | REF=7f7fffff | MODE=IEEE_ninf
# COMPARE @1600 | a=04d2bf1f b=8cd67602 | DUT=80800000 | status=28 | REF=80800000 | MODE=IEEE_ninf
# COMPARE @1610 | a=24fdf6ff b=5dfbcb1e | DUT=4379caac | status=20 | REF=4379caac | MODE=IEEE_ninf
# COMPARE @1620 | a=4d8ab94c b=67d30325 | DUT=75e4b0db | status=20 | REF=75e4b0db | MODE=IEEE_ninf
# COMPARE @1630 | a=91bb5c8a b=db56244c | DUT=2d9cb9eb | status=20 | REF=2d9cb9eb | MODE=IEEE_ninf
# COMPARE @1640 | a=25d8520e b=765951a8 | DUT=5cb7a28d | status=20 | REF=5cb7a28d | MODE=IEEE_ninf
```

Figure 6: 1ο testbench : IEEE_ninf (10 συνδυασμοί)

```
# Rounding Case : near_up
# ================================================================================================
# COMPARE @2050 | a=e5869d78 b=ffb79104 | DUT=7f800000 | status=82 | REF=7f800000 | MODE=near_up
# COMPARE @2060 | a=77d6ec2a b=b0242b36 | DUT=e889d392 | status=20 | REF=e889d392 | MODE=near_up
# COMPARE @2070 | a=180bda04 b=6622ddd2 | DUT=3eb1f249 | status=20 | REF=3eb1f249 | MODE=near_up
# COMPARE @2080 | a=d4b94bc6 b=84faefa2 | DUT=1a35a172 | status=20 | REF=1a35a172 | MODE=near_up
# COMPARE @2090 | a=1f88c061 b=a8f30778 | DUT=8901d299 | status=20 | REF=8901d299 | MODE=near_up
# COMPARE @2100 | a=b46900c0 b=5ad6f975 | DUT=cfc3a9ad | status=20 | REF=cfc3a9ad | MODE=near_up
# COMPARE @2110 | a=516d2b39 b=9a2dff6a | DUT=ac2132d6 | status=20 | REF=ac2132d6 | MODE=near_up
# COMPARE @2120 | a=8e4d806d b=ad5edcf6 | DUT=00000000 | status=29 | REF=00000000 | MODE=near_up
# COMPARE @2130 | a=5f2942eb b=c3a07e52 | DUT=e3543ab0 | status=20 | REF=e3543ab0 | MODE=near_up
# COMPARE @2140 | a=2da544c4 b=af3dbf0a | DUT=9d74fe33 | status=20 | REF=9d74fe33 | MODE=near_up
```

Figure 7: 1ο testbench : near_up (10 συνδυασμοί)

```
# Rounding Case : away_zero
# ================================================================================================
# COMPARE @2550 | a=4b28a9c3 b=a9c971f9 | DUT=b584b862 | status=20 | REF=b584b862 | MODE=away_zero
# COMPARE @2560 | a=79f126b1 b=63c8dace | DUT=7f800000 | status=32 | REF=7f800000 | MODE=away_zero
# COMPARE @2570 | a=1a886cdb b=82173b83 | DUT=80800000 | status=28 | REF=80800000 | MODE=away_zero
# COMPARE @2580 | a=eef4686d b=730bd6a2 | DUT=ff800000 | status=32 | REF=ff800000 | MODE=away_zero
# COMPARE @2590 | a=a1fa3587 b=4ee60f5e | DUT=b160db1d | status=20 | REF=b160db1d | MODE=away_zero
# COMPARE @2600 | a=0e38f818 b=9ebc2472 | DUT=80800000 | status=28 | REF=80800000 | MODE=away_zero
# COMPARE @2610 | a=4824c6fa b=5d05b08d | DUT=65ac1a07 | status=20 | REF=65ac1a07 | MODE=away_zero
# COMPARE @2620 | a=9d7a235d b=b5017aa4 | DUT=12fd074e | status=20 | REF=12fd074e | MODE=away_zero
# COMPARE @2630 | a=cfe53bb7 b=15ae60e9 | DUT=a61c255e | status=20 | REF=a61c255e | MODE=away_zero
# COMPARE @2640 | a=ad6894ef b=37fab60b | DUT=a5e3c6d6 | status=20 | REF=a5e3c6d6 | MODE=away_zero
```

Figure 8: 1ο testbench : away_zero (10 συνδυασμοί)

6.2 Results of second testbench

```
#  Combinations for negative signaling NaN
# COMPARE @3050 | a=ffbf01c9 b=ff970b2f | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3060 | a=ff866427 b=7f8f6669 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3070 | a=ff96d775 b=ffe872b7 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3080 | a=ffa8971b b=7fe707b4 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3090 | a=ff8cd6c1 b=ff800000 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3100 | a=ffa7b4af b=7f800000 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3110 | a=ffbd2353 b=9110ba36 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3120 | a=ffadac73 b=6f8d0709 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3130 | a=ffbdacb3 b=faddb8eb | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3140 | a=ffade42f b=1294b743 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3150 | a=ff8e20e5 b=80000000 | DUT=7f800000 | status=06 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3160 | a=ffbba6fb b=00000000 | DUT=7f800000 | status=06 | REF=7f800000 | MODE=IEEE_near
```

Figure 9: 2ο testbench : Συνδυασμοί με ένα negative singaling NaN (12 συνδυασμοί)

```
#  Combinations for positive signaling NaN
# COMPARE @3230 | a=7f9beb57 b=ff8b4d2d | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3240 | a=7fb3e54f b=7f98deb5 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3250 | a=7f9a2ba3 b=ffe64478 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3260 | a=7fa86104 b=7fee3c72 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3270 | a=7f8c8bfb b=ff800000 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3280 | a=7fac7ccf b=7f800000 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3290 | a=7fb0a299 b=f63771d9 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3300 | a=7fb8741e b=036dafb2 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3310 | a=7fb7c29c b=81adaa87 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3320 | a=7f845fa3 b=3dd8fbd3 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3330 | a=7f877919 b=80000000 | DUT=7f800000 | status=06 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3340 | a=7f9cf087 b=00000000 | DUT=7f800000 | status=06 | REF=7f800000 | MODE=IEEE_near
```

Figure 10: 2ο testbench : Συνδυασμοί με ένα positive singaling NaN (12 συνδυασμοί)

```
#  Combinations for negative quiet NaN
# COMPARE @3410 | a=ffc04155 b=ffa43e86 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3420 | a=ffd6365a b=7f98cda5 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3430 | a=ffe9bef8 b=ffdbbd31 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3440 | a=fff1f2d8 b=7ff1635a | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3450 | a=fffd3acf b=ff800000 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3460 | a=ffea2166 b=7f800000 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3470 | a=ffdd5de6 b=be9239a5 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3480 | a=ffea36c2 b=1757d2c1 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3490 | a=ffd1dac0 b=c66f6672 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3500 | a=ffe0872b b=1a51b474 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3510 | a=fff24e93 b=80000000 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3520 | a=ffe7fc8e b=00000000 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
```

Figure 11: 2ο testbench : Συνδυασμοί με ένα negative quiet NaN (12 συνδυασμοί)

```
#  Combinations for positive quiet NaN
# COMPARE @3590 | a=7fec2741 b=ffb464b0 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3600 | a=7fee6c37 b=7f8a8b4b | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3610 | a=7fc8dd5d b=fff57b9a | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3620 | a=7ff3ce2f b=7fc7c3c8 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3630 | a=7fd65327 b=ff800000 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3640 | a=7fc9ca85 b=7f800000 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3650 | a=7fff7c92 b=fbb3e89a | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3660 | a=7ffbc500 b=19e0b4d5 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3670 | a=7fe59664 b=e856a0d7 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3680 | a=7ffe12c8 b=7e958505 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3690 | a=7fc738ee b=80000000 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3700 | a=7ff51653 b=00000000 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
```

Figure 12: 2ο testbench : Συνδυασμοί με ένα positive quiet NaN (12 συνδυασμοί)

```
#  Combinations for negative infinity
# COMPARE @3770 | a=ff800000 b=ffb9ac04 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3780 | a=ff800000 b=7f8638f4 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3790 | a=ff800000 b=ffe14928 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3800 | a=ff800000 b=7fe477f3 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3810 | a=ff800000 b=ff800000 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3820 | a=ff800000 b=7f800000 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3830 | a=ff800000 b=d8a201ed | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3840 | a=ff800000 b=0da252b2 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3850 | a=ff800000 b=cbce694d | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3860 | a=ff800000 b=663bb8a5 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3870 | a=ff800000 b=80000000 | DUT=7f800000 | status=06 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3880 | a=ff800000 b=00000000 | DUT=7f800000 | status=06 | REF=7f800000 | MODE=IEEE_near
```

Figure 13: 2ο testbench : Συνδυασμοί με ένα negative infinity (12 συνδυασμοί)

```
#  Combinations for positive infinity
# COMPARE @3950 | a=7f800000 b=ffba19d5 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3960 | a=7f800000 b=7fa03716 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3970 | a=7f800000 b=fff6ac0f | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @3980 | a=7f800000 b=7fca768f | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @3990 | a=7f800000 b=ff800000 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @4000 | a=7f800000 b=7f800000 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4010 | a=7f800000 b=9b1f01e2 | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @4020 | a=7f800000 b=58b32949 | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4030 | a=7f800000 b=a7cb4b0b | DUT=ff800000 | status=02 | REF=ff800000 | MODE=IEEE_near
# COMPARE @4040 | a=7f800000 b=30a175bb | DUT=7f800000 | status=02 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4050 | a=7f800000 b=80000000 | DUT=7f800000 | status=06 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4060 | a=7f800000 b=00000000 | DUT=7f800000 | status=06 | REF=7f800000 | MODE=IEEE_near
```

Figure 14: 2ο testbench : Συνδυασμοί με ένα positive infinity (12 συνδυασμοί)

```
#  Combinations for negative normal
# COMPARE @4130 | a=b108f232 b=ffbb9bcb | DUT=7f800000 | status=82 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4140 | a=9d093d93 b=7fb525d5 | DUT=ff800000 | status=82 | REF=ff800000 | MODE=IEEE_near
# COMPARE @4150 | a=e0e5d2be b=ffc1c575 | DUT=7f800000 | status=82 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4160 | a=fb33494f b=7ff4ead8 | DUT=ff800000 | status=82 | REF=ff800000 | MODE=IEEE_near
# COMPARE @4170 | a=dfdd7798 b=ff800000 | DUT=7f800000 | status=82 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4180 | a=bda9cd5a b=7f800000 | DUT=ff800000 | status=82 | REF=ff800000 | MODE=IEEE_near
# COMPARE @4190 | a=8314906f b=ccbcc4db | DUT=105b189e | status=20 | REF=105b189e | MODE=IEEE_near
# COMPARE @4200 | a=d84673b8 b=57a93513 | DUT=f0832b89 | status=20 | REF=f0832b89 | MODE=IEEE_near
# COMPARE @4210 | a=d5e31a29 b=b9045fe7 | DUT=4f6add22 | status=20 | REF=4f6add22 | MODE=IEEE_near
# COMPARE @4220 | a=ea177c1c b=2ae508cb | DUT=d5878739 | status=20 | REF=d5878739 | MODE=IEEE_near
# COMPARE @4230 | a=cd6c72c9 b=80000000 | DUT=00000000 | status=01 | REF=00000000 | MODE=IEEE_near
# COMPARE @4240 | a=ee555d31 b=00000000 | DUT=80000000 | status=01 | REF=80000000 | MODE=IEEE_near
```

Figure 15: 2ο testbench : Συνδυασμοί με ένα negative normal (12 συνδυασμοί)

```
#  Combinations for positive normal
# COMPARE @4310 | a=1b569816 b=ffb77df9 | DUT=ff800000 | status=82 | REF=ff800000 | MODE=IEEE_near
# COMPARE @4320 | a=5907b658 b=7f941cc3 | DUT=7f800000 | status=82 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4330 | a=2a05560c b=ffec0f44 | DUT=ff800000 | status=82 | REF=ff800000 | MODE=IEEE_near
# COMPARE @4340 | a=27063271 b=7fe1b3b7 | DUT=7f800000 | status=82 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4350 | a=2a24d633 b=ff800000 | DUT=ff800000 | status=82 | REF=ff800000 | MODE=IEEE_near
# COMPARE @4360 | a=32dfd79e b=7f800000 | DUT=7f800000 | status=82 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4370 | a=600a3cb2 b=ed375a1a | DUT=ff800000 | status=32 | REF=ff800000 | MODE=IEEE_near
# COMPARE @4380 | a=773f86c3 b=26d31689 | DUT=5e9decef | status=20 | REF=5e9decef | MODE=IEEE_near
# COMPARE @4390 | a=27157d7f b=cd19b58d | DUT=b4b3840a | status=20 | REF=b4b3840a | MODE=IEEE_near
# COMPARE @4400 | a=040d9fb4 b=69a491d1 | DUT=2e3615f5 | status=20 | REF=2e3615f5 | MODE=IEEE_near
# COMPARE @4410 | a=0f28bd7b b=80000000 | DUT=80000000 | status=01 | REF=80000000 | MODE=IEEE_near
# COMPARE @4420 | a=5b74aac6 b=00000000 | DUT=00000000 | status=01 | REF=00000000 | MODE=IEEE_near
```

Figure 16: 2ο testbench : Συνδυασμοί με ένα positive normal (12 συνδυασμοί)

```
#  Combinations for negative denormal
# COMPARE @4490 | a=803bd3ea b=ffa9b2e7 | DUT=7f800000 | status=82 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4500 | a=8001b050 b=7f8b12c5 | DUT=7f800000 | status=86 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4510 | a=8049c4ee b=ffef7698 | DUT=7f800000 | status=86 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4520 | a=802c848d b=7fde710d | DUT=7f800000 | status=86 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4530 | a=806b5b7e b=ff800000 | DUT=7f800000 | status=82 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4540 | a=801bb72b b=7f800000 | DUT=7f800000 | status=86 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4550 | a=80124255 b=835aec32 | DUT=00000000 | status=01 | REF=00000000 | MODE=IEEE_near
# COMPARE @4560 | a=802964ef b=05825297 | DUT=80000000 | status=01 | REF=80000000 | MODE=IEEE_near
# COMPARE @4570 | a=8078d03a b=9535799f | DUT=00000000 | status=01 | REF=00000000 | MODE=IEEE_near
# COMPARE @4580 | a=803f75f1 b=608a178b | DUT=80000000 | status=01 | REF=80000000 | MODE=IEEE_near
# COMPARE @4590 | a=80317591 b=80000000 | DUT=00000000 | status=01 | REF=00000000 | MODE=IEEE_near
# COMPARE @4600 | a=8073a91c b=00000000 | DUT=80000000 | status=01 | REF=80000000 | MODE=IEEE_near
```

Figure 17: 2ο testbench : Συνδυασμοί με ένα negative denormal (12 συνδυασμοί)

```
#  Combinations for positive denormal
# COMPARE @4670 | a=0026c10c b=ffa743ce | DUT=7f800000 | status=82 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4680 | a=003b4147 b=7f8bb924 | DUT=7f800000 | status=82 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4690 | a=000903b3 b=ffec7422 | DUT=7f800000 | status=82 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4700 | a=0004f915 b=7fe45ab1 | DUT=7f800000 | status=82 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4710 | a=003975b1 b=ff800000 | DUT=7f800000 | status=86 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4720 | a=0019ab95 b=7f800000 | DUT=7f800000 | status=86 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4730 | a=0051b01b b=a5169c8b | DUT=80000000 | status=01 | REF=80000000 | MODE=IEEE_near
# COMPARE @4740 | a=000c6ff4 b=02a54d3c | DUT=00000000 | status=01 | REF=00000000 | MODE=IEEE_near
# COMPARE @4750 | a=00671eee b=d12e12f3 | DUT=80000000 | status=01 | REF=80000000 | MODE=IEEE_near
# COMPARE @4760 | a=0036bc32 b=26e24f2a | DUT=00000000 | status=01 | REF=00000000 | MODE=IEEE_near
# COMPARE @4770 | a=007f298a b=80000000 | DUT=80000000 | status=01 | REF=80000000 | MODE=IEEE_near
# COMPARE @4780 | a=002eaa09 b=00000000 | DUT=00000000 | status=01 | REF=00000000 | MODE=IEEE_near
```

Figure 18: 2ο testbench : Συνδυασμοί με ένα positive denormal (12 συνδυασμοί)

```
#  Combinations for negative zero
# COMPARE @4850 | a=80000000 b=ff9716fb | DUT=7f800000 | status=86 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4860 | a=80000000 b=7fb3a755 | DUT=7f800000 | status=86 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4870 | a=80000000 b=ffc668e2 | DUT=7f800000 | status=82 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4880 | a=80000000 b=7fe15854 | DUT=7f800000 | status=82 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4890 | a=80000000 b=ff800000 | DUT=7f800000 | status=86 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4900 | a=80000000 b=7f800000 | DUT=7f800000 | status=86 | REF=7f800000 | MODE=IEEE_near
# COMPARE @4910 | a=80000000 b=ba904b0d | DUT=00000000 | status=01 | REF=00000000 | MODE=IEEE_near
# COMPARE @4920 | a=80000000 b=15206f45 | DUT=80000000 | status=01 | REF=80000000 | MODE=IEEE_near
# COMPARE @4930 | a=80000000 b=dbd222bb | DUT=00000000 | status=01 | REF=00000000 | MODE=IEEE_near
# COMPARE @4940 | a=80000000 b=26c713cc | DUT=80000000 | status=01 | REF=80000000 | MODE=IEEE_near
# COMPARE @4950 | a=80000000 b=80000000 | DUT=00000000 | status=01 | REF=00000000 | MODE=IEEE_near
# COMPARE @4960 | a=80000000 b=00000000 | DUT=80000000 | status=01 | REF=80000000 | MODE=IEEE_near
```

Figure 19: 2ο testbench : Συνδυασμοί με ένα negative zero (12 συνδυασμοί)

```
#  Combinations for positive zero
# COMPARE @5030 | a=00000000 b=ffb2a19e | DUT=7f800000 | status=86 | REF=7f800000 | MODE=IEEE_near
# COMPARE @5040 | a=00000000 b=7fb8f42c | DUT=7f800000 | status=86 | REF=7f800000 | MODE=IEEE_near
# COMPARE @5050 | a=00000000 b=ffcb28e6 | DUT=7f800000 | status=82 | REF=7f800000 | MODE=IEEE_near
# COMPARE @5060 | a=00000000 b=7fed0b03 | DUT=7f800000 | status=82 | REF=7f800000 | MODE=IEEE_near
# COMPARE @5070 | a=00000000 b=ff800000 | DUT=7f800000 | status=86 | REF=7f800000 | MODE=IEEE_near
# COMPARE @5080 | a=00000000 b=7f800000 | DUT=7f800000 | status=86 | REF=7f800000 | MODE=IEEE_near
# COMPARE @5090 | a=00000000 b=866c9a8c | DUT=80000000 | status=01 | REF=80000000 | MODE=IEEE_near
# COMPARE @5100 | a=00000000 b=6c29e600 | DUT=00000000 | status=01 | REF=00000000 | MODE=IEEE_near
# COMPARE @5110 | a=00000000 b=9f24c2a4 | DUT=80000000 | status=01 | REF=80000000 | MODE=IEEE_near
# COMPARE @5120 | a=00000000 b=0ce1496c | DUT=00000000 | status=01 | REF=00000000 | MODE=IEEE_near
# COMPARE @5130 | a=00000000 b=80000000 | DUT=80000000 | status=01 | REF=80000000 | MODE=IEEE_near
# COMPARE @5140 | a=00000000 b=00000000 | DUT=00000000 | status=01 | REF=00000000 | MODE=IEEE_near
```

Figure 20: 2ο testbench : Συνδυασμοί με ένα positive zero (12 συνδυασμοί)

7 System Verilog Assertions (SVA)

We proceed to the final stage of the assignment concerning the assertions we need to implement. All assertions are contained in the file "sv_assertions.sv". Including this file along with the testbench in subsequent simulations is recommended for completeness (both print out much better together). We distinguish between two types of assertions: immediate and concurrent.

For the immediate assertions, we are required to implement assertions for all pairs of status bits that cannot simultaneously assert to 1. This is not particularly demanding, provided we identify them. Let us examine all possible combinations to determine which ones need to be verified.

The combinations involving zero_f are:

- zero_f and inf_f : simultaneous assertion not allowed
- zero_f and nan_f : simultaneous assertion not allowed
- zero_f and tiny_f : simultaneous assertion allowed
- zero_f and huge_f : simultaneous assertion not allowed
- zero_f and inexact_f : simultaneous assertion allowed
- zero_f and unused_f : simultaneous assertion not allowed
- zero_f and div_by_0 _f : simultaneous assertion not allowed

The remaining combinations involving inf_f are:

- inf_f and nan_f : simultaneous assertion allowed
- inf_f and tiny_f : simultaneous assertion not allowed
- inf_f and huge_f : simultaneous assertion allowed
- inf_f and inexact_f : simultaneous assertion allowed
- inf_f and unused_f : simultaneous assertion not allowed
- inf_f and div_by_0 _f : simultaneous assertion allowed

The remaining combinations involving nan_f are:

- nan_f and tiny_f : simultaneous assertion not allowed
- nan_f and huge_f : simultaneous assertion not allowed
- nan_f and inexact_f : simultaneous assertion not allowed
- nan_f and unused_f : simultaneous assertion not allowed
- nan_f and div_by_0 _f : simultaneous assertion allowed (not considered rounding)

The remaining combinations involving tiny_f are:

- tiny_f and huge_f : simultaneous assertion not allowed
- tiny_f and inexact_f : simultaneous assertion allowed
- tiny_f and unused_f : simultaneous assertion not allowed
- tiny_f and div_by_0 _f : simultaneous assertion not allowed

The remaining combinations involving huge_f are:

- huge_f and inexact_f : simultaneous assertion allowed
- huge_f and unused_f : simultaneous assertion not allowed
- huge_f and div_by_0 _f : simultaneous assertion not allowed

The remaining combinations involving inexact_f are:

- inexact_f and unused_f : simultaneous assertion not allowed
- inexact_f and div_by_0 _f : simultaneous assertion not allowed

The single remaining combination involving unused_f is:

- unused_f and div_by_0 _f : simultaneous assertion not allowed

In total, we count 19 immediate assertions. When these pass, we print “PASS” to the transcript, otherwise “FAIL”. These are contained within the module test_status_bits. We ensure that this module is bound to the wrapper in the testbench.

The concurrent assertions check on every positive edge of the clock that specific conditions are satisfied provided certain premises hold. It is probably of little interest to report further details, as the process of defining a property and using assert property is quite straightforward. The only detail we will mention is that in the assertion where we need to verify whether “a” and “b” have exponents such that one equals zero (all zeros) and the other equals 255 (all ones), we utilized the useful $past() function configured to check three cycles back. These are described in test_status_z_combinations, and binding to the wrapper is performed once again in the testbench.

We know that all assertions are properly satisfied because during the testbench “FAIL” is printed nowhere. We will show some, but running the simulation is again recommended.

```
#  @590 inf_f && unused_f : PASS
#  @590 nan_f && tiny_f : PASS
#  @590 nan_f && huge_f : PASS
#  @590 nan_f && inexact_f : PASS
#  @590 nan_f and unused_f : PASS
#  @590 tiny_f && huge_f : PASS
#  @590 tiny_f && unused_f : PASS
#  @590 tiny_f && div_by_0_f : PASS
#  @590 huge_f && unused_f : PASS
#  @590 huge_f && div_by_0_f : PASS
#  @590 inexact_f && unused_f : PASS
#  @590 inexact_f && div_by_0_f : PASS
#  @590 unused_f && div_by_0_f : PASS
#  @590 huge_f and z : PASS
#  @600 zero_f && inf_f : PASS
#  @600 zero_f && nan_f : PASS
#  @600 zero_f && huge_f : PASS
#  @600 zero_f && unused_f : PASS
#  @600 zero_f && div_by_0_f : PASS
#  @600 inf_f && nan_f : PASS
#  @600 inf_f && unused_f : PASS
#  @600 nan_f && tiny_f : PASS
#  @600 nan_f && huge_f : PASS
#  @600 nan_f && inexact_f : PASS
#  @600 nan_f and unused_f : PASS
#  @600 tiny_f && huge_f : PASS
#  @600 tiny_f && unused_f : PASS
#  @600 tiny_f && div_by_0_f : PASS
#  @600 huge_f && unused_f : PASS
#  @600 huge_f && div_by_0_f : PASS
#  @600 inexact_f && unused_f : PASS
#  @600 inexact_f && div_by_0_f : PASS
#  @600 unused_f && div_by_0_f : PASS
#  @600 tiny_f and z : PASS
#  @600 zero_f and z_exponent : PASS
#  @630 zero_f && inf_f : PASS
#  @630 zero_f && nan_f : PASS
#  @630 zero_f && huge_f : PASS
#  @630 zero_f && unused_f : PASS
#  @630 zero_f && div_by_0_f : PASS
```

Figure 21: Messages from assertions (1st image)

```
#  @4710 inf_f && nan_f : PASS
#  @4710 inf_f && unused_f : PASS
#  @4710 nan_f && tiny_f : PASS
#  @4710 nan_f && huge_f : PASS
#  @4710 nan_f && inexact_f : PASS
#  @4710 nan_f and unused_f : PASS
#  @4710 tiny_f && huge_f : PASS
#  @4710 tiny_f && unused_f : PASS
#  @4710 tiny_f && div_by_0_f : PASS
#  @4710 huge_f && unused_f : PASS
#  @4710 huge_f && div_by_0_f : PASS
#  @4710 inexact_f && unused_f : PASS
#  @4710 inexact_f && div_by_0_f : PASS
#  @4710 unused_f && div_by_0_f : PASS
#  @4710 inf_f and z_exponent : PASS
#  @4720 nan_f and a_exponent and b_exponent : PASS
#  @4720 inf_f and z_exponent : PASS
#  @4730 zero_f && inf_f : PASS
#  @4730 zero_f && nan_f : PASS
#  @4730 zero_f && huge_f : PASS
#  @4730 zero_f && unused_f : PASS
#  @4730 zero_f && div_by_0_f : PASS
#  @4730 inf_f && nan_f : PASS
#  @4730 inf_f && unused_f : PASS
#  @4730 nan_f && tiny_f : PASS
#  @4730 nan_f && huge_f : PASS
#  @4730 nan_f && inexact_f : PASS
#  @4730 nan_f and unused_f : PASS
#  @4730 tiny_f && huge_f : PASS
#  @4730 tiny_f && unused_f : PASS
#  @4730 tiny_f && div_by_0_f : PASS
#  @4730 huge_f && unused_f : PASS
#  @4730 huge_f && div_by_0_f : PASS
#  @4730 inexact_f && unused_f : PASS
#  @4730 inexact_f && div_by_0_f : PASS
#  @4730 unused_f && div_by_0_f : PASS
#  @4730 nan_f and a_exponent and b_exponent : PASS
#  @4730 inf_f and z_exponent : PASS
#  @4740 zero_f and z_exponent : PASS
#  @4750 zero_f and z_exponent : PASS
#  @4760 zero_f and z_exponent : PASS
#  @4770 zero_f and z_exponent : PASS
#  @4780 zero_f and z_exponent : PASS
#  @4790 zero_f and z_exponent : PASS
```

Figure 22: Messages from assertions (2nd image)

```
#  @4820 unused_f && div_by_0_f : PASS
#  @4830 zero_f and z_exponent : PASS
#  @4840 zero_f and z_exponent : PASS
#  @4850 zero_f && inf_f : PASS
#  @4850 zero_f && nan_f : PASS
#  @4850 zero_f && huge_f : PASS
#  @4850 zero_f && unused_f : PASS
#  @4850 zero_f && div_by_0_f : PASS
#  @4850 inf_f && nan_f : PASS
#  @4850 inf_f && unused_f : PASS
#  @4850 nan_f && tiny_f : PASS
#  @4850 nan_f && huge_f : PASS
#  @4850 nan_f && inexact_f : PASS
#  @4850 nan_f and unused_f : PASS
#  @4850 tiny_f && huge_f : PASS
#  @4850 tiny_f && unused_f : PASS
#  @4850 tiny_f && div_by_0_f : PASS
#  @4850 huge_f && unused_f : PASS
#  @4850 huge_f && div_by_0_f : PASS
#  @4850 inexact_f && unused_f : PASS
#  @4850 inexact_f && div_by_0_f : PASS
#  @4850 unused_f && div_by_0_f : PASS
#  @4850 zero_f and z_exponent : PASS
#  @4860 nan_f and a_exponent and b_exponent : PASS
#  @4860 inf_f and z_exponent : PASS
#  @4870 zero_f && inf_f : PASS
#  @4870 zero_f && nan_f : PASS
#  @4870 zero_f && huge_f : PASS
#  @4870 zero_f && unused_f : PASS
#  @4870 zero_f && div_by_0_f : PASS
#  @4870 inf_f && nan_f : PASS
#  @4870 inf_f && unused_f : PASS
#  @4870 nan_f && tiny_f : PASS
#  @4870 nan_f && huge_f : PASS
#  @4870 nan_f && inexact_f : PASS
#  @4870 nan_f and unused_f : PASS
#  @4870 tiny_f && huge_f : PASS
#  @4870 tiny_f && unused_f : PASS
#  @4870 tiny_f && div_by_0_f : PASS
#  @4870 huge_f && unused_f : PASS
#  @4870 huge_f && div_by_0_f : PASS
#  @4870 inexact_f && unused_f : PASS
#  @4870 inexact_f && div_by_0_f : PASS
#  @4870 unused_f && div_by_0_f : PASS
#  @4870 nan_f and a_exponent and b_exponent : PASS
```

Figure 23: Messages from assertions (3rd image)